Real-World DevOps Project

Azure DevOps + GitOps + IaC

Automating deployments across Dev, Staging and Production

CI/CD | GitOps | Infrastructure as Code

Complete Project Guide

Contents: Synopsis, Architecture, 10-Day Plan, Code Samples, Suggested Add-ons, Documentation Checklist, Demo and Viva Guide

Duration: 10 days

Tools: Azure DevOps, Terraform, AKS, ACR, Argo CD, Kustomize

Table of Contents

1. Project Synopsis
2. Architecture and Workflow
3. Technology Stack and Prerequisites
4. Repository Strategy
5. 10-Day Implementation Plan (Day 1 to Day 10)
6. Core Code Samples
7. Suggested Add-ons to Make the Project Stronger
8. Documentation Checklist
9. Cost Control, Risks and Troubleshooting
10. Demo Script, Viva Questions and Resume Points

1. Project Synopsis

Field	Details
Title	Automated Multi-Environment Deployment using Azure DevOps, GitOps and Infrastructure as Code
Domain	DevOps, Cloud Engineering, Platform Engineering
Duration	10 days
Cloud	Microsoft Azure
Environments	Development, Staging, Production
Key tools	Azure DevOps Repos and Pipelines, Terraform, Docker, AKS, ACR, Argo CD, Kustomize, Key Vault, Prometheus, Grafana

1.1 Introduction

Traditional deployments rely on manual steps, which lead to configuration drift, inconsistent environments, slow releases and human error. This project builds a fully automated delivery platform. Infrastructure is created from code (Terraform), applications are built and tested by CI pipelines (Azure Pipelines), and deployments are driven from Git using GitOps (Argo CD). Git becomes the single source of truth for both infrastructure and application state.

1.2 Problem Statement

- Manual infrastructure creation is slow, undocumented and not repeatable.
- Dev, staging and production environments drift apart over time.
- Pipelines that push directly to the cluster need powerful credentials and give no clear audit trail.
- Rollbacks are slow and risky without version-controlled state.
- Secrets are often hard-coded or shared insecurely.

1.3 Objectives

- Provision all Azure infrastructure (VNet, ACR, AKS, Key Vault) using reusable Terraform modules.
- Build a CI pipeline that builds, tests, scans and pushes a container image on every commit.
- Implement GitOps with Argo CD so the cluster always matches the Git repository.
- Automate promotion across dev, staging and prod with approval gates.
- Secure the platform with Key Vault, RBAC, network policies and image scanning.
- Add monitoring, alerting, drift detection and a tested rollback process.

1.4 Scope

In scope: IaC, CI, GitOps CD, multi-environment promotion, secrets management, basic observability, documentation. **Out of scope:** multi-cloud, service mesh, large-scale performance tuning (listed as future enhancements).

1.5 Methodology

Phased approach: foundation (days 1-2), infrastructure (days 3-5), CI and GitOps (days 6-8), hardening and observability (day 9), validation and presentation (day 10).

1.6 Expected Outcomes

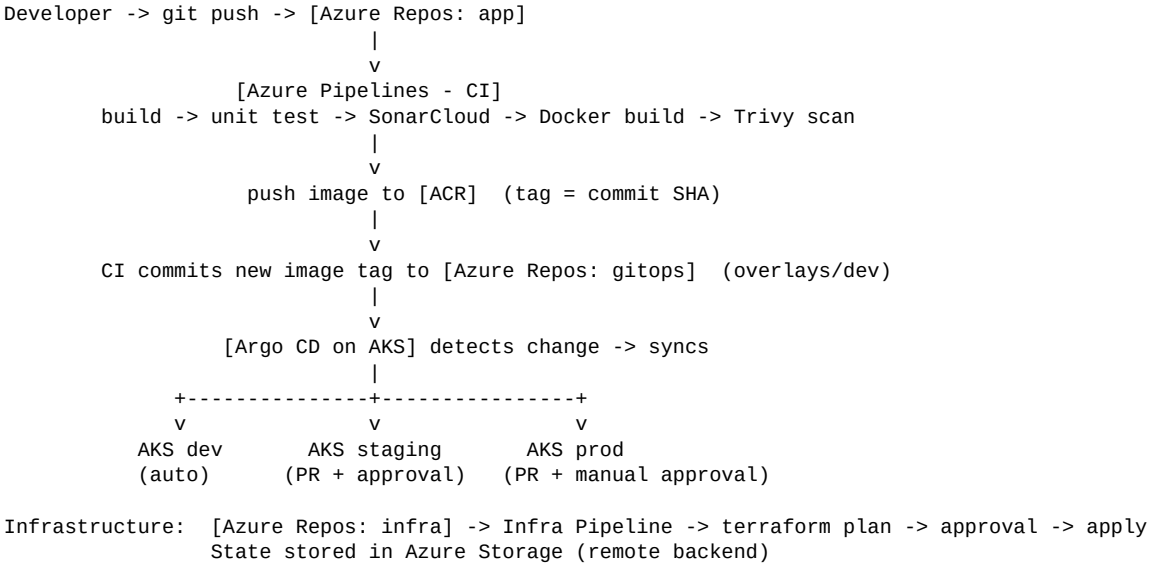
- Environments provisioned or destroyed in minutes with one pipeline run.
- Code commit to running dev deployment with no manual steps.
- Auditable, pull-request-based promotion to staging and production.
- Automatic self-healing when someone changes the cluster manually (drift correction).
- One-command rollback using git revert.

1.7 Conclusion

The project demonstrates a production-style DevOps platform that combines CI/CD, GitOps and IaC, improving speed, reliability, security and traceability of software delivery.

2. Architecture and Workflow

End-to-end flow:



Key principles

- **Pull-based deployment:** Argo CD inside the cluster pulls from Git; the CI pipeline never needs cluster admin credentials.
- **Separation of concerns:** app code, infrastructure code and deployment manifests live in separate repositories.
- **Immutable images:** images are tagged with the commit SHA, never 'latest'.
- **Environment promotion:** the same image moves dev -> staging -> prod; only config changes via overlays.

Promotion policy

Environment	Trigger	Approval	Sync policy
Dev	Automatic after CI success	None	Auto-sync, self-heal, prune
Staging	PR from dev overlay change	1 reviewer + checks	Auto-sync after merge
Prod	PR from staging	2 reviewers / manual gate	Manual sync or auto after merge

3. Technology Stack and Prerequisites

Layer	Tool	Purpose
Source control	Azure Repos (Git)	Three repositories, branch policies, PRs
CI	Azure Pipelines (YAML)	Build, test, scan, push image
IaC	Terraform	VNet, ACR, AKS, Key Vault, remote state
Container	Docker (multi-stage)	Package the application
Registry	Azure Container Registry	Store images
Orchestration	Azure Kubernetes Service	Run workloads
GitOps	Argo CD + Kustomize	Declarative continuous delivery
Secrets	Azure Key Vault + CSI driver	Secure secret injection
Quality/Security	SonarCloud, Trivy	Code quality and vulnerability scanning
Observability	Prometheus, Grafana / Azure Monitor	Metrics, dashboards, alerts

Prerequisites checklist (already installed)

- Azure subscription (free credit or student account) and Azure DevOps organization
- az CLI, Terraform (1.5+), kubectl, Helm, Git, Docker Desktop, VS Code
- Optional: argocd CLI, kustomize, Trivy

Quick verification

```
az --version && terraform -version && kubectl version --client
helm version && git --version && docker --version
az login && az account show
```

4. Repository Strategy

```
app/
  src/
  tests/
  Dockerfile
  azure-pipelines.yml
  sonar-project.properties
```

```
infra/
  modules/
    network/
    acr/
    aks/
    keyvault/
  envs/
    dev.tfvars
    staging.tfvars
    prod.tfvars
  main.tf variables.tf
  backend.tf outputs.tf
  infra-pipeline.yml
```

```
gitops/
  base/
    deployment.yaml
    service.yaml
    kustomization.yaml
  overlays/
    dev/kustomization.yaml
    staging/kustomization.yaml
    prod/kustomization.yaml
  argocd-apps/
    dev-app.yaml staging-app.yaml
    prod-app.yaml
```

Branch policy: main is protected; changes via PR; minimum 1 reviewer; build validation required; linked work items.

5. 10-Day Implementation Plan

Day	Phase	Focus	Key deliverable
1	Foundation	Azure + DevOps setup, repos, tools	Architecture diagram, repos ready
2	Foundation	Sample app + Docker	Containerized app with tests
3	IaC	Terraform backend, network, ACR	Remote state + base infra
4	IaC	AKS, Key Vault, env tfvars	Dev cluster running
5	IaC	Infra pipeline with approvals	All 3 envs via pipeline
6	CI	App CI pipeline, scans	Image in ACR with SHA tag
7	GitOps	Argo CD, Kustomize overlays	Auto-sync working
8	CD	CI to GitOps, promotion flow	Dev auto, staging/prod via PR
9	Hardening	Secrets, RBAC, monitoring, rollback	Security + alerts + runbook
10	Validation	End-to-end test, docs, demo	Final report + demo

Day 1: Planning and Setup

Goal: Prepare accounts, tools, repositories and design.

Tasks

- Create Azure DevOps organization and project; enable Repos and Pipelines.
- Create repos: app, infra, gitops. Initialize each with README and .gitignore.
- Verify tool installation and run az login.
- Create a service principal / service connection (Project Settings > Service connections > Azure Resource Manager).
- Set branch policies on main (PR required, 1 reviewer, build validation).
- Draw the architecture diagram (draw.io) and define naming convention, e.g. rg-devops-dev, aks-devops-dev, acrdevops.

Key commands / snippets

```
az login
az account set --subscription "<SUBSCRIPTION_ID>"
az ad sp create-for-rbac --name sp-devops-project --role Contributor \
   --scopes /subscriptions/<SUBSCRIPTION_ID>
```

Deliverable: Repos created, service connection verified, architecture diagram.

Documentation to write: ARCHITECTURE.md (diagram, repo strategy, naming conventions).

Day 2: Containerize the Application

Goal: Have a tested, containerized app ready for the pipelines.

Tasks

- Build a small REST app (Node.js/Express or Python/Flask) with /, /health and /version endpoints.
- Add 2-3 unit tests.
- Write a multi-stage Dockerfile; run as a non-root user.
- Run locally and test endpoints; commit to the app repo.

Key commands / snippets

```
# Dockerfile (Node.js example)
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm test

FROM node:20-alpine
WORKDIR /app
COPY --from=build /app .
USER node
EXPOSE 3000
CMD ["node", "src/index.js"]

docker build -t demo-app:local . && docker run -p 3000:3000 demo-app:local
curl localhost:3000/health
```

Deliverable: Working container image running locally.

Documentation to write: README.md with local run and test steps.

Day 3: Terraform Foundation

Goal: Create remote state and base networking/registry using modules.

Tasks

- Create resource group, storage account and container for Terraform state (az CLI, one time).
- Write backend.tf pointing to the storage account.
- Create modules: network (VNet, subnets), acr.
- Run terraform init, validate, fmt, plan, apply for dev.

Key commands / snippets

```
az group create -n rg-tfstate -l centralindia
az storage account create -n sttfstate<unique> -g rg-tfstate -l centralindia --sku Standard_LRS
az storage container create -n tfstate --account-name sttfstate<unique>

terraform init
terraform plan -var-file=envs/dev.tfvars
terraform apply -var-file=envs/dev.tfvars
```

Deliverable: Remote state working; VNet and ACR created.

Documentation to write: Module guide: inputs, outputs and usage of each module.

Day 4: AKS, Key Vault and Environments

Goal: Provision a working AKS cluster and prepare per-environment configuration.

Tasks

- Create aks module: system node pool, managed identity, attach ACR (AcrPull role), enable OIDC and workload identity.
- Create keyvault module with RBAC authorization.
- Write dev.tfvars, staging.tfvars, prod.tfvars with different sizes/counts.
- Apply dev; fetch credentials and verify nodes.

Key commands / snippets

```
az aks get-credentials -g rg-devops-dev -n aks-devops-dev
kubectl get nodes
```

Deliverable: Dev AKS cluster running.

Documentation to write: Environment matrix (node size, count, region, SKU for each env).

Day 5: Infrastructure Pipeline

Goal: Automate Terraform with approvals.

Tasks

- Create infra-pipeline.yml with stages: Validate (fmt, validate), Plan (publish plan artifact), Approve (ManualValidation / Environment approval), Apply.
- Create Azure DevOps Environments: infra-dev, infra-staging, infra-prod; add approvals to prod.
- Run pipeline for staging and prod.

Key commands / snippets

```
stages:
- stage: Validate
  jobs:
  - job: v
    steps:
    - script: terraform init -backend=false && terraform fmt -check && terraform validate
- stage: Plan
```

```
    dependsOn: Validate
    jobs:
    - job: p
      steps:
      - script: terraform plan -var-file=envs/$(env).tfvars -out=tfplan
  - stage: Apply
    dependsOn: Plan
    jobs:
    - deployment: apply
      environment: infra-$(env)     # approval configured on environment
      strategy:
        runOnce:
          deploy:
            steps:
            - script: terraform apply -auto-approve tfplan
```

Deliverable: Staging and prod provisioned through the pipeline.

Documentation to write: IaC pipeline flow diagram and approval rules.

Day 6: CI Pipeline for the Application

Goal: Build, test, scan and publish images automatically.

Tasks

- Create azure-pipelines.yml triggered on main.
- Stages: Build and Test, SonarCloud analysis, Docker build, Trivy scan, Push to ACR.
- Tag images with $(Build.SourceVersion) short SHA.
- Fail pipeline on HIGH/CRITICAL vulnerabilities.

Key commands / snippets

```
trigger: [ main ]
variables:
  imageRepo: demo-app
  tag: $(Build.SourceVersion)
steps:
- script: npm ci && npm test
  displayName: Build and test
- task: Docker@2
  inputs:
    command: build
    repository: $(imageRepo)
    tags: $(tag)
- script: trivy image --exit-code 1 --severity HIGH,CRITICAL acr.azurecr.io/$(imageRepo):$(tag)
- task: Docker@2
  inputs:
    command: push
    containerRegistry: acr-service-connection
    repository: $(imageRepo)
    tags: $(tag)
```

Deliverable: Image with SHA tag in ACR after every commit.

Documentation to write: CI stages and quality gate description.

Day 7: GitOps Setup with Argo CD

Goal: Make the cluster pull its desired state from Git.

Tasks

- Install Argo CD on AKS using Helm.
- Create gitops repo structure: base/ and overlays/dev|staging|prod (Kustomize).
- Add deployment, service, ingress in base; overlays patch replicas, resources and image tag.

- Create Argo CD Application manifests per environment; connect the Azure Repos gitops repo (PAT).
- Verify auto-sync, self-heal and prune.

Key commands / snippets

```
helm repo add argo https://argoproj.github.io/argo-helm
helm install argocd argo/argo-cd -n argocd --create-namespace
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d
kubectl port-forward svc/argocd-server -n argocd 8080:443
```

Deliverable: Argo CD showing app Healthy and Synced.

Documentation to write: GitOps repo structure and sync policy.

Day 8: Connect CI to GitOps and Promotion

Goal: Complete the CI to CD loop with controlled promotion.

Tasks

- Add a final CI stage that clones the gitops repo, updates the image tag in overlays/dev (kustomize edit set image) and pushes.
- Dev deploys automatically via Argo CD.
- Promotion: pipeline or script opens a PR that updates overlays/staging; after approval, merge and Argo syncs.
- Repeat for prod with stricter approval (2 reviewers).

Key commands / snippets

```
cd gitops/overlays/dev
kustomize edit set image demo-app=acr.azurecr.io/demo-app:$(Build.SourceVersion)
git config user.email "ci@project.com" && git config user.name "CI Bot"
git commit -am "dev: update image to $(Build.SourceVersion)"
git push origin main
```

Deliverable: Commit to app repo reaches dev automatically; staging/prod via PR.

Documentation to write: Promotion workflow diagram and rules.

Day 9: Security and Observability

Goal: Harden the platform and add visibility.

Tasks

- Integrate Key Vault using the Secrets Store CSI driver (or External Secrets Operator) with workload identity.
- Add Kubernetes RBAC, namespaces per environment, NetworkPolicies, resource limits.
- Enable Azure Policy for AKS (e.g. block privileged containers).
- Install kube-prometheus-stack; create Grafana dashboard and alerts (pod restarts, high CPU, 5xx).
- Test rollback: git revert the bad commit and watch Argo CD restore the previous version.

Key commands / snippets

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm install monitoring prometheus-community/kube-prometheus-stack -n monitoring --create-namespace

git revert <bad_commit_sha> && git push    # rollback via GitOps
```

Deliverable: Secrets from Key Vault, dashboards and alerts working, rollback proven.

Documentation to write: SECURITY.md checklist and RUNBOOK.md rollback steps.

Day 10: Testing, Polish and Presentation

Goal: Validate everything and prepare the demo.

Tasks

- End-to-end test: change code -> CI -> dev -> staging PR -> prod PR.
- Drift test: kubectl scale deployment manually; Argo CD reverts it.
- Failure test: push a failing test or a vulnerable image; confirm pipeline blocks it.
- Capture screenshots; finalize README, diagrams, report.
- Record a 5-minute demo; run terraform destroy to avoid cost.

Key commands / snippets

```
kubectl scale deploy demo-app -n dev --replicas=5     # watch Argo CD revert to desired count
terraform destroy -var-file=envs/dev.tfvars
```

Deliverable: Final report, demo video, clean-up complete.

Documentation to write: Final project report with screenshots and lessons learned.

6. Core Code Samples

6.1 Terraform backend

```
terraform {
  required_providers { azurerm = { source = "hashicorp/azurerm", version = "~> 3.100" } }
  backend "azurerm" {
    resource_group_name  = "rg-tfstate"
    storage_account_name = "sttfstate<unique>"
    container_name       = "tfstate"
    key                  = "dev.terraform.tfstate"
  }
}
provider "azurerm" { features {} }
```

6.2 AKS module (simplified)

```
resource "azurerm_kubernetes_cluster" "aks" {
  name                = "aks-${var.project}-${var.env}"
  location            = var.location
  resource_group_name = var.rg_name
  dns_prefix          = "${var.project}-${var.env}"
  oidc_issuer_enabled       = true
  workload_identity_enabled = true
  default_node_pool {
    name       = "system"
    vm_size    = var.node_size
    node_count = var.node_count
  }
  identity { type = "SystemAssigned" }
}
resource "azurerm_role_assignment" "acr_pull" {
  scope                = var.acr_id
  role_definition_name = "AcrPull"
  principal_id         = azurerm_kubernetes_cluster.aks.kubelet_identity[0].object_id
}
```

6.3 Environment variables

Setting	dev	staging	prod
VM size	Standard_B2s	Standard_B2s	Standard_D2s_v3
Node count	1	2	3
App replicas	1	2	3
Sync	auto	auto after PR	PR + manual approval

6.4 Kustomize overlay (dev)

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: dev
resources:
  - ../../base
images:
  - name: demo-app
    newName: acrdevops.azurecr.io/demo-app
    newTag: abc1234
replicas:
  - name: demo-app
    count: 1
```

6.5 Argo CD Application

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: demo-app-dev
  namespace: argocd
```

```
spec:
  project: default
  source:
    repoURL: https://dev.azure.com/<org>/<project>/_git/gitops
    targetRevision: main
    path: overlays/dev
  destination:
    server: https://kubernetes.default.svc
    namespace: dev
  syncPolicy:
    automated: { prune: true, selfHeal: true }
    syncOptions: [ CreateNamespace=true ]
```

7. Suggested Add-ons to Make the Project Stronger

Pick items based on your remaining time. 'Must' items are quick wins that greatly improve the project's value.

Priority	Add-on	Why it adds value	Effort
Must	Trivy + SonarCloud quality gates	Shows DevSecOps; pipeline blocks bad code/images	Low
Must	Drift detection and self-heal demo	Proves GitOps benefit visually	Low
Must	Rollback via git revert	Strong demo point	Low
Must	Key Vault secrets via CSI / External Secrets	No secrets in Git	Medium
Should	Azure Monitor / Prometheus + Grafana alerts	Observability, production readiness	Medium
Should	Argo CD Image Updater	Alternative to CI commit step	Medium
Should	Ingress NGINX + cert-manager (HTTPS)	Real URL with TLS	Medium
Should	Terraform security scan (tfsec / Checkov)	IaC security gate in pipeline	Low
Should	Infracost in PR	Cost visibility before apply	Low
Nice	Helm chart instead of Kustomize	Shows packaging skills	Medium
Nice	Blue/green or canary with Argo Rollouts	Advanced release strategy	High
Nice	HPA autoscaling and load test (k6)	Performance and scaling demo	Medium
Nice	Slack / Teams notifications from pipelines and Argo CD	Team workflow realism	Low
Nice	Policy as code with OPA Gatekeeper / Kyverno	Governance	Medium
Nice	Backup with Velero, multi-region DR	Resilience	High

Recommended 'extra credit' bundle if time is short

- Trivy + tfsec gates
- Key Vault secrets
- Prometheus/Grafana dashboard
- Ingress with HTTPS
- Slack notifications

Future enhancements to mention in your report

- Multi-cluster / multi-region deployment with Argo CD ApplicationSets
- Service mesh (Istio/Linkerd) for mTLS and traffic control
- FinOps dashboards and automated environment shutdown schedules
- Platform engineering portal (Backstage) for self-service environments

8. Documentation Checklist

File	Content
README.md	Project overview, quick start, repo links, screenshots
ARCHITECTURE.md	Diagram, components, data flow, design decisions
INFRASTRUCTURE.md	Terraform modules, variables, environment matrix
CICD.md	CI and infra pipeline stages, quality gates, variables
GITOPS.md	Repo structure, sync policy, promotion workflow
SECURITY.md	Secrets, RBAC, network policy, scanning, checklist
RUNBOOK.md	Deploy, rollback, scale, incident steps
TROUBLESHOOTING.md	Common errors and fixes
FINAL_REPORT.pdf	Synopsis, implementation, results, screenshots, conclusion

Screenshots to capture

- Azure DevOps repos and branch policy
- Successful CI run with all stages
- Trivy / Sonar results
- ACR repository with tagged images
- Terraform plan and apply output
- Argo CD dashboard (3 apps Healthy/Synced)
- PR approval for staging/prod
- Drift correction event
- Grafana dashboard
- Rollback result

9. Cost Control, Risks and Troubleshooting

Cost control

- Use B-series small nodes and 1-2 nodes for dev/staging.
- Use the AKS free tier; stop clusters when idle (az aks stop) or terraform destroy after each session.
- Set an Azure budget alert at a low threshold.
- Use ACR Basic SKU.

Risks and mitigation

Risk	Mitigation
Unexpected Azure cost	Budget alerts, destroy daily, small SKUs
Parallelism limit on free Azure DevOps	Request free parallelism or use a self-hosted agent
Terraform state corruption	Remote state with locking, versioning on the blob container
Secrets leaked to Git	Key Vault, secret scanning, .gitignore, never commit tfvars with secrets

Risk	Mitigation
Time overrun	Build the must-have flow first; add extras only after the end-to-end demo works

Common issues

Problem	Fix
ImagePullBackOff	Check AcrPull role on kubelet identity; verify image name and tag
Argo CD OutOfSync loop	Check ignoreDifferences, resource defaults, and kustomize output
Pipeline cannot push to gitops repo	Grant Contribute permission to the build service; use PAT or System.AccessToken
Terraform state lock	terraform force-unlock after confirming no run is active
No hosted parallelism	Submit the Microsoft free-tier request form or use a self-hosted agent

10. Demo Script, Viva Questions and Resume Points

5-minute demo flow

- Show architecture diagram (30 s).
- Show the three repos and branch policies (30 s).
- Change a line in the app and push (30 s).
- Show CI stages, scans and image in ACR (60 s).
- Show Argo CD syncing dev automatically (45 s).
- Open PR for staging, approve, show sync (60 s).
- Scale a deployment manually and show self-heal (30 s).
- Show rollback by git revert (30 s).

Likely viva / interview questions

Question	Short answer
What is GitOps?	Operating model where Git is the single source of truth and an agent in the cluster continuously reconciles actual state to the Git state.
Why Argo CD (pull) rather than pipeline push?	No cluster credentials in CI, automatic drift correction, full audit trail in Git.
Why a separate gitops repo?	Separates app build history from deployment state; cleaner access control and promotion.
Why Terraform remote state?	Shared, locked, versioned state so teams and pipelines can safely collaborate.
How do you promote to prod?	A PR updates the prod overlay with the already-tested image tag; approvals gate the merge.
How do you roll back?	git revert the commit; Argo CD syncs the previous manifest.
How are secrets handled?	Stored in Key Vault and mounted via CSI driver / External Secrets using workload identity.
Why tag images with SHA?	Immutable, traceable to the exact commit, avoids 'latest' ambiguity.

Resume bullet points

- Designed and implemented a multi-environment CI/CD platform on Azure using Azure DevOps, Terraform, AKS and Argo CD (GitOps).
- Provisioned infrastructure as reusable Terraform modules with remote state and approval-gated pipelines.
- Automated build, test, SonarCloud and Trivy scanning, reducing manual release steps to zero for dev.
- Implemented PR-based promotion, drift detection/self-healing and Git-based rollback.
- Secured secrets with Azure Key Vault and workload identity; added Prometheus/Grafana monitoring.

End of document. Keep this guide in the repository as docs/PROJECT_GUIDE.pdf.